

Search-Eliminating Computation — The Method of Complete Structural Construction

**A Non-Search Approach to Discrete Constraint Satisfaction
Problems: Proposal and Implementation**

目次

Abstract

1. Introduction

2. Design Philosophy

3. The Four-Element Framework of A

4. Information Sources and the Division of Roles Between Two Computations

5. Program-Level Implementation

6. The Boundary Between Structural Propagation and Search

7. Performance Characteristics

8. Circuit Level: Physicalizing the Structure

9. Physical Verification

10. Discussion

11. Limits and Future Work

Appendix A: Reproduction Procedure

Appendix B: Recommended and Prohibited Items at the Circuit Level

Appendix C: Key Terms

Abstract

Most computational methods for discrete constraint satisfaction problems place search — wandering through a space of candidate solutions — at their core. This paper proposes a computational mode that stands opposite to this. That is: a non-search paradigm in which the structure of the solution is completely constructed in advance, and the solution is reached solely through the chain of determinations that this structure forces.

We first confirmed that this description in terms of four elements holds for a broad range of discrete NP-hard problems, requiring no new descriptive patterns across domains. We then implemented this mode at the program level, showing that more than 20 kinds of discrete NP-hard problems can be handled within a unified framework, and that its behavior can be backed by fresh measurement over a body of 1500 problems. Each problem is classified by solution type into six kinds, all processed by the same core mechanism. This core is composed of a computation that carries out structural propagation (Φ -computing) and a separate computation that scrutinizes and inspects its run (λ -computing). The implementation adopts a design that thoroughly excludes any determination path lacking structural grounds, and as a result it clearly separates "structurally legitimate success" from "honest halting that lacks legitimate grounds." Furthermore, we showed quantitatively that for problems where the forcing chain reliably propagates, the computational cost is linear in problem size, and that for problems lacking a forcing origin, the system halts as designed.

This paper shows that this is not merely a software trick but a physical circuit design principle, demonstrating it through implementation and measurement. The value of this paradigm lies not in speed or breadth as a general-purpose solver, but in determining solutions without search, with the determined results being structurally legitimate (zero false success), and in halting honestly when determination is not forced.

1. Introduction

1.1 Background

Computational methods for discrete constraint satisfaction problems and discrete NP-hard problems are, for the most part, founded on search. Conflict-driven learning in satisfiability, backtracking in constraint programming, and stochastic local search in hardware solvers all share the trait of traversing a space of candidate solutions by some policy. While these handle a broad range of problems, as long as they presuppose search, their worst-case complexity can be exponential in problem size.

What this paper asks lies in the opposite direction. That is: rather than searching for the solution, can we construct the structure of the solution completely in advance, so that the solution emerges solely through the chain of determinations that the structure forces — thereby making search itself unnecessary?

1.2 Idea

In certain constraint satisfaction problems, there are consequences uniquely fixed by the initially given constraints. Implication from a unit clause, and exclusion by a capacity constraint, are typical examples, and none of them involve "choice." When such forced determinations exist, they propagate like a chain, each determination forcing the next. If this chain reaches the entire solution, then the solution is determined without any search.

What matters here is that whether the chain reaches the whole, or stops partway, is itself determined by the structure of the problem. This paradigm does not paper over a stopped chain by guessing. Where the chain stops, it stops — and it shows that fact honestly.

1.3 Contributions

The contributions of this paper are fourfold. First, we formalize the design philosophy of describing the structure of a solution as four constituent elements and eliminating search through their complete construction (Chapters 2 and 3). Second, we show the separation of information sources and responsibilities that supports this philosophy, and implement it as a unified framework applicable to more than 20 kinds of discrete NP-hard problems at the program level (Chapters 4 and 5). Third, we honestly distinguish, as separate computational mechanisms, the range solvable by structural propagation alone from the range requiring search (Chapter 6). Fourth, we quantify the performance characteristics and, further, give a proof-of-concept of the implementability of this mode as a circuit through physical simulation (Chapters 7–9).

2. Design Philosophy

2.1 Allowing No False Success

At the root of this paradigm's design philosophy lies a discipline: "allow no false success."

An implementation of constraint satisfaction often, when it hits a wall, flees into existing computational methods. Such operations include introducing search as an aid, trying other candidates by backtracking, retrying stochastically, or importing the machinery of existing solvers. These raise apparent solving power, but at that moment the paradigm ceases to be "computation that does not search."

This paradigm forbids these at the design stage. When the chain of determination does not reach the whole, the cause lies in the construction of the structure — that is, in the quality of the description. We call this the "quality debt of A." Even when there is debt, the system does not rescue itself by branching, backtracking, retrying, or importing existing computational methods. It leaves the debt as debt, and honestly shows that it cannot determine.

2.2 Build the Structure and Search Disappears

From the discipline above follows the central idea of this paradigm. That is: construct the structure of the solution completely in advance, and search disappears.

Ordinary computation "chooses one from several candidates, and backtracks if it errs." Because there is choice and backtracking, this becomes search. Against this, this paradigm builds the structure of the solution as four elements, and lets only the chain of determinations forced by that structure run. Where forcing reaches, determination is fixed; where forcing does not reach, the system stops there. It does not choose. Because it forbids choice, this paradigm is in principle incapable of search.

2.3 Design Principles

To secure the philosophy above, this paradigm explicitly prescribes prohibitions. It has none of: branching, backtracking, recursion, guessing hidden facts, importing the core of existing solvers, running rescue plugins, undo or restart, or forcibly fixing values. Each of these amounts to either "search" or "flight into existing computational methods." By structurally excluding these, we guarantee that this paradigm is neither search nor a rephrasing of existing solvers.

3. The Four-Element Framework of A

3.1 The Four Constituent Elements

This paradigm describes the structure of a solution as four constituent elements. These are: the self-position (A_{self}), which expresses where the current determination stands; the goal (A_{goal}), which expresses the conditions the solution should satisfy; the line of sight to the goal (A_{view}), which expresses the path of determinations connecting the present to the goal; and the overshoot ($A_{\text{overshoot}}$), which expresses deviations and excesses to be corrected.

These four are not an arbitrary decomposition. They correspond to the powers required to reach a goal without searching: the power to set a goal, the power to run toward it, the power to keep the goal in view, and the power to adjust when one has run too far. When all four are prepared before computation, what remains is only to run — and search becomes unnecessary.

3.2 The Computational Properties the Four Elements Decide

This four-element description decides the computational properties of the paradigm. When the four elements are completely constructed, the chain of determinations forced by the structure reaches the whole, and the solution is determined without search. When the four elements are incompletely constructed, the chain stops partway, and the system halts there. There is no middle ground of "search to fill the gap."

3.3 Robustness of the Framework

To confirm that this four-element description is not a device peculiar to a particular problem, we examined the describability in terms of four elements for discrete NP-hard problems across a broad range of domains — logic and satisfiability, graph structure, paths and tours, sets and assignment, placement, numeric selection, puzzles, and so on. Problems in every domain could be described as four elements, and across domains no new descriptive pattern needed to be added. The framework is closed across problem domains. This claim of describability is backed by the fresh measurement over a body of 1500 problems, actually run, described from the next chapter onward.

At the same time, it became clear that the framework has a boundary of applicability. In domains such as two-player perfect-information games, real-time strategy, problems mediated by physical simulation, and multi-agent coordination, even though description as four elements is itself possible, fixing the structure of the solution in advance is in principle difficult, and the inherent strength does not show. In adversarially branching games in particular, the solution is not a fixed structure but a strategy tree dependent on the opponent's replies, so the premise of advance construction does not hold. This paradigm does not hide this boundary. We state explicitly that these domains lie outside the honest range of application at present. Making the boundary explicit, if anything, raises the credibility of the claims.

4. Information Sources and the Division of Roles Between Two Computations

4.1 The Composition of Two Computations

The core of this paradigm is composed of two computations with different roles: a computation that carries out structural propagation (Φ -computing), and a separate computation that scrutinizes and inspects its run (λ -computing). This is the greatest feature distinguishing this paradigm from existing computational modes.

We call the computational system to which this paradigm belongs κ -computing. κ -computing is composed of the computation that determines through structural propagation (Φ), the computation that inspects its run (λ), and the computation that generates hypotheses ($\mathcal{M}$). Of these, hypothesis generation ($\mathcal{M}$) belongs only to a higher level of abstraction and is not treated in this paper. Φ and λ both exist at the program level and cooperate as separate computations.

What matters here is that κ -computing is not a third solver with a fixed order of execution. κ -computing is the very boundary that keeps two equal computations (Φ and λ) within the same runtime, ensuring that neither strays into search or existing methods.

4.2 The Three Dimensions of Information Sources

The information that Φ uses to construct the structure comes from three dimensions. The parallel-present dimension (AC): past experience, present information, initial conditions one can gather at hand. The concurrent dimension (AJ): cooperation with other sources, multiple paths, reverse-walking from the goal — multicore, parallel computation, quantum computing belong here. The higher dimension (AH): algorithm-level meta-information acquisition such as net search, and direct input of information by the user.

At the program level, AH is weak. Therefore AC (information at hand) and AJ (cooperation and reverse-walking) must compensate for the missing powers. This paradigm constructs the four elements of A by drawing primarily on AC and AJ.

4.3 Φ : Running and Structural Observation

What Φ carries out is fixing determination through structural propagation. Φ stays within three constraints: building the structure of the solution, not searching, and compressing computation instead of widening a tree of candidates.

Φ runs a single trajectory, and where the forcing chain reaches, it fixes determination; where it does not reach, it halts. If the chain does not propagate, it is the quality debt of A, and Φ does not rescue itself by branching, retrying, or importing existing-solver machinery. Φ does not use the weakness of A as a pretext to take in another computational mode.

4.4 λ : Scrutiny, Inspection, and Human Intervention

λ is a separate computation that inspects Φ 's run. λ stays within three roles: inspection, the origin of learning, and human intervention.

Inspection: scrutinizes Φ 's halting situations and structurally diagnoses why it cannot advance. This role is pure observation with no side effects; it neither rewrites records nor changes external state. It only judges the structural cause of a halt.

Learning: receiving the result of diagnosis, it updates the logic interpreting determinations, outside the run. This role specializes in side effects and does not engage in the judgment itself. Each time the run halts, the structure being grasped is gradually strengthened by this role.

Human intervention: opens, to a human outside the run, the points where determination has halted as origins of learning. The human carries out final judgment and decisions.

Coordination: binds the above three roles, mediating when and which role to invoke against Φ 's run.

4.5 Connection, Not Mixing

That Φ and λ are separate computations raises the question of how to connect them. This paradigm does not blend them together but connects them at limited points.

Legitimate connection is limited to two points. First, the connection from run to inspection — after Φ fixes a determination, or after determination halts, λ reads its trace. Second, the connection from inspection to human — the origin of learning that λ has opened is received by a human outside the run.

Against this, there are mixings that are forbidden: turning λ 's observation into automatic commands during the run, treating the origin of learning as an immediate repair policy, bringing higher-level machinery in during the run, and filling gaps with the existing computational mode of another solver. Each of these breaks the separation of running and inspection, and drags this paradigm back toward search or existing solvers. The two are kept as separate computations and connected only at prescribed points.

4.6 Accumulation of Experience and Purity

The traces of past determinations are recorded, but that record stays a pure log. The recording part neither infers nor corrects within; it only returns "this is how the past was," and never says "therefore you should do this." Interpretation is borne by λ outside the run. By this separation, we prevent the accumulation of experience from sliding into "substantive learning" and creeping into Φ 's run.

Here, what should be accumulated requires care. If a concrete determination value obtained on a particular problem is carried over as-is to another problem, then because variable values are specific to each problem, it can collide with the structural facts of the other problem and produce a false contradiction. This danger was confirmed in measurement as well. What may be accumulated and fed back is not concrete determination values, but only the structural implications that hold in common across a problem family — value-independent implication relations of the form "if this condition holds, that consequence is forced." When only structural implications were fed back, no false contradiction arose, and determinations on new problems increased. Accumulation of experience is not memory of values, but accumulation of structural facts.

5. Program-Level Implementation

5.1 Unified Framework and Coverage of Problems

We implemented this paradigm at the program level and composed it as a unified framework for discrete NP-hard problems. The targets span more than 20 kinds, including satisfiability, graph coloring, Sudoku, knapsack, subset sum, vertex cover, set cover, independent set, max cut, clique, dominating set, tours, Hamiltonian cycle,

Steiner tree, capacitated vehicle routing, quadratic assignment, and various scheduling.

These problems are classified by solution type into six kinds: path type, subset type, permutation type, assignment type, partial-constraint-satisfaction type, and continuous-value type. The framework covers these six and applies the same core mechanism to every type. What is prepared per problem is only the structural knowledge specific to that problem (which determinations are forced, which deviations to detect); the skeleton of solving is common.

5.2 The Division of Responsibility in Solving

In this framework, what is prepared per problem and the common core are clearly separated. Per problem, only the generation of structural knowledge — enumerating which determinations are forced, and detecting which deviations are excesses — is given. The core that receives this and lets the chain of determinations propagate is common to all problems. Solving is carried out not as an independent solver per problem, but as feeding structural knowledge into the common propagation loop.

5.3 Structurally Legitimate Success and Honest Halting

We verified the validity of this framework over a body of 1500 problems. We classified each problem into success structurally determined, unresolved halted for lack of forcing, and contradiction structurally proven, and counted them. Furthermore, for those judged as successes, we verified in full whether the solution was truly valid. The 183 cases determined in satisfiability were all correct as satisfying assignments, and the 273 cases determined in Sudoku all passed the row, column, and box rules; there was not a single "false success" lacking structural grounds. We confirmed through full verification that the situation in which what is displayed as "solved" is in fact wrong never occurs even once.

6. The Boundary Between Structural Propagation and Search

6.1 Two Computational Mechanisms

This paradigm clearly separates the range solvable by structural propagation alone from the range requiring search, treating them as two separate computational mechanisms. Structural propagation fixes determination only where forcing reaches; where forcing does not reach, it halts. Search is the act of choosing candidates and backtracking — which this paradigm does not perform. By treating the two as separate mechanisms, this paradigm does not let search creep into structural propagation.

6.2 Honest Limitation of Range

The range of structural propagation is limited to where forcing reaches. This paradigm does not hide this limitation. Where the forcing chain does not complete — problems lacking a forcing origin, situations where freedom of choice remains — the system

halts. This is not a defect but a design behavior: where determination is not forced, not determining is correct.

6.3 The Positioning of Complementary Mechanisms

For situations where structural propagation does not reach, search-based complementary mechanisms can in principle be combined. But that is the work of a separate mechanism, distinct from this paradigm. This paradigm does not internalize a complementary mechanism. It keeps the boundary, and connects, if at all, at prescribed points — never mixing.

7. Performance Characteristics

7.1 The Boundary of Strengths and Weaknesses

The boundary of this paradigm's strengths and weaknesses appears at a single point: whether a forcing origin exists. We measured Sudoku at each clue count over 100 trials each: solving rate 0% at 30 clues, 30% at 40, 83% at 50, 94% at 55, and 98% at 60. This threshold rises not as a sharp cliff but continuously. Because the difficulty of individual problems varies with random placement, the aggregate becomes a gentle curve. The boundary of strength appears where the amount of initial constraint crosses a certain band.

Yet beneath this aggregate gentleness hide the sharp criticalities of individual problems. In measurements removing clues one by one from the same solution, each problem shows a sharp critical point of "here the forcing chain stops completing," and that value varied per problem in the range of 45 to 52 clues. The gentleness of the aggregate curve is the averaged appearance of individual sharp criticalities lying at different positions per problem.

This behavior is not a defect but an expression of the very nature of the paradigm. If constraint is strong, the forcing chain reaches the whole; if weak, the chain stops partway. When the chain stops, because this paradigm does not search, it halts there. What matters is that what divides strength from weakness is not the kind of problem, but the single point of whether a forcing origin exists and its chain can complete the whole solution. For the same problem, given a forcing origin it is solved; not given one, it halts.

The same tendency is consistent across other problems. For random satisfiability without unit clauses to serve as forcing origins, the determination rate was 0% in all 2100 trials varying variable count and clause ratio; each clause was evaluated once and it halted immediately, determining not a single variable. On the other hand, in placement problems the solving rate rises with more forcing origins. In N-Queens, increasing the clues extracted from a solution, the six-board reached full solution at four clues, the eight-board at six, and the ten-board at eight. The larger the board, the more origins required. In graph coloring, coloring an even-length cycle in two colors completes without search through uniquely transmitted forcing, while for an odd-

length cycle it structurally proved that two-coloring is impossible as a contradiction. On the other hand, where freedom of color remains (three or more colors with one origin), it halted without guessing.

All of these show that strength and weakness are decided not by the kind of problem but by the presence of a forcing origin. Placement problems (N-Queens) too halt without an origin and are solved by collapse with one. It is not whether the problem is "puzzle or logic," but whether forcing can complete the chain, that decides everything.

7.2 Linearity of Propagation Cost

For problems where the forcing chain reliably propagates, the number of propagation operations required for determination is linear in problem size. Expanding an implication chain up to size one million, the number of operations per size was constant (4.00 per size), and the complexity was strictly proportional to problem size.

This linearity is not limited to a linear chain. For any structure where forcing reaches the whole — tree-shaped implication (511 nodes at depth 8), a directed acyclic graph where multiple paths merge (900,000-variable scale), a set of Horn clauses (one-million scale) — all variables were determined, and operations per size stayed constant. The propagation of forcing does not choose the shape of the problem's structure.

7.3 Honest Comparison with Existing Solvers

This paradigm does not outdo general-purpose solvers in sequential wall-clock time, nor in breadth of coverage. Mature SAT solvers, by conflict-driven learning and stochastic local search, solve, in practical time, even large problems where this paradigm halts for lack of a forcing origin. We state this honestly. The value of this paradigm lies elsewhere — in determining without search, deterministically, order-independently, linearly, and traceably, within the range where forcing reaches, with the solution being structurally legitimate.

8. Circuit Level: Physicalizing the Structure

This paradigm is implemented not only at the program level but also as a hardware circuit. The circuit level realizes structural propagation as collapse propagation in a physical circuit, showing that this paradigm's mode of determination is not peculiar to software but a physical circuit design principle.

8.1 The Collapse-Type Circuit Mode

At the circuit level, determination is realized as the "collapse" of holding elements. A holding element holds a state, and when forcing is injected, it collapses irreversibly to the determined value. The collapse of one element forces the collapse of the next, and this propagates like a chain — the physical incarnation of structural propagation.

8.2 Composition

The circuit is composed of holding elements (cells that hold and collapse states), forcing-injection paths, and collapse-propagation wiring. There is neither a central controller nor a max-selector. Determination is fixed when collapse propagates from the forcing origin.

8.3 Design Norms

The circuit follows the same norms as the program level. It has no search, no backtracking, no central selection. Collapse propagates only where forcing reaches, and halts where it does not. Program-level structural propagation and circuit-level collapse are different implementations of the same mode.

9. Physical Verification

9.1 Premises of Verification

To verify that the circuit operates as intended, we conducted circuit simulation at the conceptual-PDK level. This is a conceptual element model and is not an absolute physical performance (process, layout, post-synthesis timing, silicon area, measured power are all out of scope). What this chapter verifies is the existence of parallel collapse, not its performance on silicon.

9.2 Observed Parallel Collapse

A single holding element collapsed irreversibly upon forcing injection, and did not return even after release. Connecting two elements, forcing the origin caused the next element to collapse automatically through the propagation path. Connecting five holding elements in a chain and forcing collapse only at the origin, all five stages collapsed within an extremely short time after forcing injection (nearly simultaneous within the time resolution). Placing multiple chains and forcing collapse at each origin simultaneously, the ends of two chains both collapsed at the same time instant, confirming that parallel collapse from multiple origins occurs on a physical circuit.

We also verified the case where determination is not binary. With a holding element that can take four colors, forbidding three colors left only the remaining one color, and multi-valued forced determination held as a circuit. Furthermore, expressing in a circuit the structure where both colors are forbidden in trying to two-color an odd-length cycle, a node indicating contradiction rose, and the coloring-impossibility was detected as a physical phenomenon as a structural contradiction.

9.3 Significance and Limits of Observation

These observations show that this paradigm's mode of determination holds as a physical circuit. Structural propagation runs not as a software abstraction but as a signal on the circuit. However, the verification stays at the conceptual element-model and small scale; parallelism at large scale (hundreds of stages or more), the physical

behavior of more complex cells, and the effects of temperature and variation are all unverified. What this chapter claims is the existence of parallel collapse, not its scaling limit.

10. Discussion

10.1 Where the Value of This Paradigm Lies

The value of this paradigm lies neither in speed nor in coverage as a general-purpose solver. It lies in three things. First, the computational mode itself: completely constructing the structure of the solution, and fixing forcing without choice or backtracking. Second, clearly separating "structurally legitimate success" from "honest halting," and honestly suspending determination in situations where determination is not forced. Third, that this mode holds in both software and hardware, and can be implemented as a physical circuit design principle.

10.2 Structural Legitimacy of the Output

The second point matters for uses where reliability is at stake. This paradigm fixes determination only where the forcing chain reaches, and where freedom remains it holds without choosing. Thereby, the determinations this paradigm fixes are limited to those forced by structure, not chosen by guesswork. For problems it cannot determine, it honestly suspends rather than returning a wrong determination. Full verification backs that the fixed determinations are correct. There is, here, a design choice not to sacrifice legitimacy in exchange for solving power.

10.3 On the Idea of Complete Structural Construction

The problem families this paradigm treats — satisfiability, graph coloring, tours, clique, knapsack, and so on — are all problem groups deemed hard in complexity theory. Against these, this paradigm consistently uses the fact that when the structure of the solution can be completely constructed, search becomes unnecessary. That the boundary of "can determine / stays unforced" is expressed as the completability of the forcing chain itself offers one view on where the difficulty of computation resides. How far its theoretical implications can be generalized is a problem requiring broader consideration beyond the scope of this paper.

11. Limits and Future Work

11.1 Limits

The determination of this paradigm is limited in range to situations where the forcing chain reaches the whole. Where a forcing origin is lacking, or where freedom of choice remains, determination does not occur, and this paradigm holds these without choosing. This is a design behavior, but uses requiring determination need combination with other means. The circuit implementation stays at the hardware-description and reference-model stage; process-level element design, physical

placement and routing, post-synthesis timing, silicon area, and measured power are untouched.

This paradigm is not a general-purpose method that solves all hard problems in polynomial time. Nor does this paper claim a proof regarding the fundamental problem of complexity theory. What this paper shows is a concrete implementation that determines forcing as complete structural construction, without using search.

11.2 Future Work

There are three directions. First, to characterize more precisely, per problem family, the quality of structural construction — the ability to correctly express the structure of the solution as four elements. Second, to advance the circuit implementation to synthesis, place-and-route, and the silicon level, clarifying its physical performance as dedicated hardware. Third, integration with another, complementary line of approach. If a domain that structural propagation does not reach can be filled by another mode, combining the two could greatly broaden the solvable problems. In particular, a sister paradigm that treats the same problem group with a different mode of determination — the compression of a pressure field — is a promising partner, in that it could fix, from a different angle, situations this paradigm holds as contention. This integration is best done in a form that connects the two while keeping their boundary, after each mode is independently established; we treat it in a separate paper.

Appendix A: Reproduction Procedure

The program-level implementation of this paradigm is composed of a common propagation loop and per-problem structural knowledge. Solving each problem is carried out by giving the loop the structural knowledge specific to that problem (enumeration of forced determinations, detection of deviations). Verification is by the procedures of the core benchmark, the load benchmark, full solution validation, and linearity measurement. Circuit-level verification is by the procedure of constructing, on a simulator, a circuit of holding elements and propagation paths, and observing collapse propagation from a forcing origin. The tools and input formats needed for reproduction are provided as a reproduction guide and written-out input files.

Appendix B: Recommended and Prohibited Items at the Circuit Level

Item	Recommended / Prohibited
Collapse propagation from a forcing origin	Recommended
Parallel collapse from multiple origins	Recommended
Multi-valued collapse cell	Recommended
Contradiction detection node	Recommended
Central controller / max-selector	Prohibited
Search / backtracking circuit	Prohibited

Forcible value-fixing without grounds	Prohibited
---------------------------------------	------------

Appendix C: Key Terms

κ -computing: the general name for the computational system of this paper. Composed of Φ -computing, λ -computing, and $\mathcal{M}$ -computing; not a third solver with a fixed order of execution, but the boundary keeping two equal computations within the same runtime. **Φ -computing**: the computation that realizes determination as structural propagation. The protagonist of this paper. Does not search, does not flee into existing computational methods. **λ -computing**: the computation that inspects Φ 's run. Performs only inspection, the origin of learning, and human intervention; does not feed results back into the run (read-only). **$\mathcal{M}$ -computing**: the computation that generates hypotheses. Belongs only to a higher level of abstraction and is not treated in this paper. **The four elements of A**: self-position, goal, line of sight to the goal, and overshoot. **Forcing origin**: a structure that initiates a forced determination. **Quality debt of A**: that determination does not complete because the structural construction is insufficient. Not compensated by search.

(Note: The proper names of existing algorithms, theorems, and solvers mentioned in this paper have been intentionally kept out of the main text to distinguish them from this paper's framework. This paper is paired with a sister paradigm that treats the same problem group with a different mode of determination. Even where terms appear shared between the two, each follows the definition within its own paradigm; the correspondence of terms is organized in a separate paper discussing integration.)